

POCKETSMART: AI-POWERED SMART BUDGET PLANNER

OFFICIAL PHASE-WISE SOFTWARE DEVELOPMENT REPORT

FRAMEWORK: FastAPI (Python ASGI Backend) & Bootstrap 5 UI Engineering

PHASE 1: BRAINSTORMING & IDEATION PHASE

- **Core Problem Statement:** Budgeting for lifestyle events like home interior modifications, wedding events, and jewelry procurement is usually scattered across fragmented offline templates or manual web searches, causing heavy budget overruns and operational human calculation errors.
- **The AI Innovation:** PocketSmart AI acts as an autonomous digital financial coordinator that captures structural limits (budgets, counts) and automatically splits them into optimized, micro-allocated categories.
- **Ecosystem Integration:** Embedded direct retail deep-links (Amazon, Flipkart, Swiggy, Zomato, CaratLane) within the output grid views to eliminate manual tracking.

PHASE 2: REQUIREMENT ANALYSIS PHASE

- **Functional Requirements:** Secure parsing forms for financial boundary enforcement, an interactive options selector board, and cross-origin query engines capable of dynamic parameters passing.
- **Non-Functional Requirements:** Asynchronous operations under 400ms loading latencies, clean navigation rendering loops, and fluid responsive styling parameters.
- **System Dependencies:** Python ASGI Environment, FastAPI Software Suite, Pydantic Schema Layers, and Bootstrap 5 Component Core.

PHASE 3: PROJECT DESIGN PHASE

- **Architectural Pattern:** Monolithic Unified Routing Architecture Paradigm.
- **Visual Style Matrix:** High-contrast crisp white cards nesting key form containers overlaying an anti-glare slate-blue viewport theme workspace configuration (#ebf2f7).
- **Route Mapping Data:** Configured clean internal template filesystem loaders to inject raw tables directly into browser documents without complex runtime storage layers.

PHASE 4: PROJECT PLANNING PHASE

- **Milestone 1 (Sprint 1):** Drafted system execution plans, built core backend server architecture in app.py, and routed base credential checkers.
- **Milestone 2 (Sprint 2):** Created frontend layout files (home_planner, party_planner, jewelry_planner) and programmed dynamic mathematical script handlers.
- **Milestone 3 (Sprint 3):** Patched CORS request security handlers, handled browser cache injection traps, and audited live hyperlink arrays.

PHASE 5: PROJECT DEVELOPMENT PHASE

- **Backend Implementation:** Coded an enterprise-grade asynchronous ASGI microserver engine using FastAPI in app.py that natively delivers clean file streams using local templates.
- **Client-Side Processing Engine:** Built isolated javascript evaluation logic layers inside the document templates to instantly evaluate budget metrics on-click without causing continuous local backend port connection strains.

PHASE 6: PROJECT TESTING PHASE

- **Test Registry Endpoint:** Local application network loopback interface framework (<http://127.0.0.1>).
- **Unit Test Case (Syntax Recovery):** Resolved nested python multi-line quote syntax literal crashes by separating front-end HTML assets out of app.py variables.
- **Integration Test Case (CORS Block Recovery):** Fixed local script blockages and "Connection Error" warnings by shifting processing execution directly into internal front-end functions.

- **Validation Test Case (URL Rectification):** Fixed a typo string inside the redirection map link array (amazon.inled+bulb) ensuring instant handshakes with global e-commerce servers.

PHASE 7: PROJECT DOCUMENTATION PHASE

- **Technical Material Hosted:** Consolidated application operations documentation, initialization setup instructions, and deployment run targets into a permanent master workflow configuration registry asset file.
- **Evidence Archiving:** Collected system performance reports and active terminal console data strings to prove operational execution logs.

PHASE 8: PROJECT DEMONSTRATION PHASE

- **Production Evaluation Workflow:** Verified that a user successfully authenticates on the landing portal, accesses the master dashboard layout configuration layer, and correctly triggers budget allocations inside Home, Party, and Jewelry pipelines with functional external shopping redirects.